

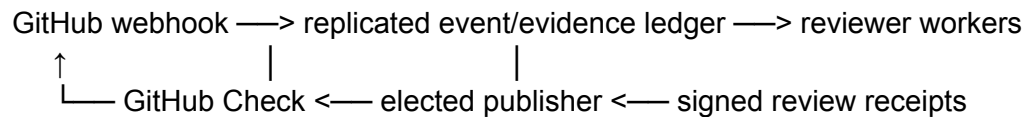
## GitHub Layer:

Yes. I'd make it a **distributed PR-governance control plane**, with GitHub as the source of truth for code and merging.

No successful GitHub Check may be published for PR epoch E **uMergeProof:**  
**Fault-Tolerant, Evidence-Bound AI Review Quorums for GitHub Pull Requests**

GitHub stays normal: developers open PRs, branch protection remains enabled, and a GitHub App posts a required Check Run on each commit. GitHub Check Runs are already attached to a specific commit SHA, which is exactly the boundary you need. [GitHub's Checks API](#)

The interesting distributed system is behind it:



Each PR revision becomes an immutable **PR epoch**:

(repo, PR number, base SHA, head SHA, policy version)

Every AI review receipt is bound to that epoch. A receipt from before a force-push, a different base branch, an old policy, or a revoked reviewer cannot turn the new PR green.

To make it research-grade, use a hybrid consistency model:

- **Highly available evidence plane:** multiple workers receive duplicated/out-of-order GitHub webhooks, invoke reviewers, and add signed receipts to a replicated event log/CRDT.
- **Strongly consistent publication plane:** a small Raft/lease-backed publisher elects one replica to update the GitHub Check. This prevents split-brain controllers from publishing contradictory results.
- **Fail closed:** if the evidence quorum or publisher lease is unavailable, the GitHub check stays pending—not incorrectly successful.
- **Risk-adaptive quorum:** documentation-only changes might need one cheap reviewer; authentication, permissions, payments, migrations, or CI changes require independent reviewers from different provider/failure domains, possibly human approval.

The central invariant becomes:

nless a current policy authorizes a valid quorum of review receipts explicitly bound to E.

That is a crisp distributed-systems claim—not “LLMs are correct.”

You can test realistic failures:

- duplicate, delayed, or reordered GitHub webhooks;
- a PR force-push while reviewers are still running;
- a worker crash after an LLM response but before persistence;
- publisher/leader failure immediately before or after posting a Check;
- provider timeout, retry, or conflicting review;
- policy update or reviewer revocation mid-review;
- regional partition between controller replicas;
- budget exhaustion while a high-risk PR still needs quorum.

Compare against a normal single GitHub bot, naive active-active reviewers, and fixed 2-of-3 quorums. Measure false-green checks, stale-review acceptance, recovery time, review latency, API cost, and completion rate during failure.

This is arguably more practical than the P2P-repository version: every team understands GitHub PRs, the demo is immediately legible, but the hard distributed systems work remains real.

Yes—you're right. I misunderstood the intended role of the blockchain.

Your design has two separate layers:

1. Semantic verification, off-chain:

LLM evaluates candidate commit → produces signed/notarized “acceptable” receipt

2. Repository-state consensus, on-chain:

validators verify receipt cryptographically → order accepted commits →

agree on the canonical branch head

The validators do not manually review the PR and do not need to understand its code. They only deterministically verify:

- the LLM evidence is authentic;
- it refers to the exact candidate commit/diff;
- it satisfies the repository's LLM policy;
- the commit was built from the current expected branch head.

Then they use PoS/BFT consensus to decide the next repository state.

The key transaction would be:

```
AcceptCommit(  
    repo_id,  
    branch = main,  
    expected_head = H,  
    candidate_commit = C,  
    llm_receipt = R  
)
```

The consensus state machine applies it only if:

`current_head == H`

`VerifyLLMReceipt(R, C, H, policy) == true`

If valid, `main` becomes `C`. If two accepted commits arrive concurrently from the same head, consensus orders them:

Initial main: H

Transaction 1: `accept H → C1` → succeeds; main becomes C1

Transaction 2: `accept H → C2` → rejected as stale

The author of `C2` must rebase or create a new merge commit against `C1`, obtain a new LLM receipt for that new candidate, and resubmit. That is how you maintain one globally agreed repository state.

So the terminology should be:

- The LLM provides an **off-chain acceptance certificate**.
- Blockchain validators reach consensus over the **state transition**, not over code quality.
- Proof-of-stake removes the need to preselect trusted validator users, but it does not remove trust in the configured LLM provider or its receipt mechanism.

The chain stores only small metadata:

- canonical branch head commit hash;
- accepted commit history;
- policy hash/version;
- hash or compact proof of the LLM receipt;
- validator/consensus state.

Actual Git objects live off-chain—in IPFS, P2P storage, or replicated Git nodes—referenced by content hash.

This is a much cleaner and stronger research idea than the reviewer-marketplace framing:

**Can externally verified AI acceptance certificates safely gate a BFT/PoS replicated state machine for decentralized Git, preserving one canonical repository history under concurrent commit proposals?**

The primary invariant becomes:

A commit becomes part of the canonical branch only if consensus orders a state transition whose LLM acceptance certificate is valid for that exact commit and its expected parent state.

The important security boundary is also clear: PoS protects against conflicting repository histories; it does not protect against an allowed LLM incorrectly certifying bad code. You can optionally require 2-of-3 LLM certificates for high-risk changes, but that is a policy decision outside the blockchain's consensus mechanism.

We have multiple ideas for our semester long distributed systems project. Here are our ideas help us decide which is best in terms of uniqueness and relevance to distributed systems concepts.

1 ) AI agents system built to dynamically spin up multiple agents that need to come to a consensus on high risk tool calls(payments, writes to db etc). overall system goal is to minimize cost and latency and have high reliability/avoid high risk bad decisions.

2) Using a blockchain based repository system along with signed LLM PR evaluations to allow for fully decentralized git repositories. It can use a system like Gitopia or GOSH, but rather than requiring consensus, users can specify allowed LLM providers for their project, and using TSL Notary the PRs can be verified as accepted/rejected by top-tier LLMs

3) Using a coordinator in an agentic system to dynamically adjust the balance of spawned subagents to ensure that tasks run in parallel do not go over budget while adapting to failed or under budget subtasks as they are returned

4) Infrastructure for AI agent system which can be safely used by industry, developing protocol for WebMCP or tool calls for the system to decide whether one tool execution should be executed at most once or retry.

## Idea #1: AI agent consensus/reliability

Related works:

- [Rethinking the Reliability of Multi-agent System: A Perspective from Byzantine Fault Tolerance](#)
- [Multi-Agent Systems: From Classical Paradigms to Large Foundation Model-Enabled Futures](#)
- The Six Sigma Agent: Achieving Enterprise-Grade Reliability in LLM Systems Through Consensus-Driven Decomposed Execution: <https://arxiv.org/pdf/2601.22290>
- Hallucination as Context Drift: Synchronization Protocols for Multi-Agent LLM Systems: <https://arxiv.org/html/2606.21666>
- Byzantine Fault-Tolerant Multi-Agent System for Healthcare: A Gossip Protocol Approach to Secure Medical Message Propagation: <https://arxiv.org/pdf/2512.17913>

We can add Byzantine Fault-Tolerant Problem to Open Source Agent Distributed Runtime, e.g. one agent is manipulated to deposit more money in one user's account)

Ref video: <https://www.youtube.com/watch?v=hD9-V56FNRI>

Open Source Agent Distributed Runtime:

<https://github.com/google/ax>

<https://github.com/temporalio>

<https://github.com/dapr/dapr-agents/tree/main>

- **The Thesis Idea: Proposed novel angle: agreement-before-commit for agentic tool calls, not vote-after-the-fact on answers.**

- Design a dynamic BFT middleware that defaults to a single-agent execution for low-risk actions but dynamically spins up a consensus quorum only when a high-risk tool call (e.g., `ExecuteFinancialTransaction` or `ModifyDatabase`) is invoked or when anomaly detection triggers.
  - **MVP Scope:** Build an interceptor in a Go-based runtime (like Google AX or Temporal) that intercepts tool executions, calculates a risk score based on the schema, and pauses the workflow to demand semantic consensus from standby agents only when the risk crosses a threshold.
- **Another angle - The Thesis Idea:** How do you achieve BFT when agents are completely decoupled and unaware of each other, interacting only through an event log or pub/sub topics?
  - Port the concept of receiver-side semantic validation (like SAC) into an asynchronous event-driven architecture. Instead of agents verifying each other's text directly, a consensus controller independently verifies the *intent* of an agent's proposed state change against the workflow's specification before committing it to the event log.
  - **MVP Scope:** Use Dapr's pub/sub mechanisms. Agent A publishes an intent to execute a side effect. Instead of executing immediately, the message sits in a queue where Agents B and C must pull it, evaluate the semantic safety of the action against a system prompt, and publish their cryptographic "votes" before the runtime commits the action.
- **"The Honest Quorum Problem: Epistemic Byzantine Fault Tolerance for Agentic Infrastructure"** (He & Yu, arXiv 2607.16109, July 2026) — formalizes almost exactly what I proposed: agentic validators can be protocol-compliant (authenticated, non-equivocating, quorum-forming) yet still endorse a semantically wrong state transition. They name this an "epistemic fault," define EBFT (epistemic BFT) with new quorum thresholds beyond the classical  $3f+1$  bound, and explicitly flag that agents sharing model weights, training data, prompts, or toolchains break the independence assumption.
- **"The Illusion of Independent Quorums: Epistemic Fault Domains and Correlated Cognitive Failures in Agentic Quorums"** (arXiv 2609.02925, Sept 2026) — a direct follow-up, digging further into correlated cognitive failure domains.
- **"When AI agents commit: cognitive serializability across data, evidence, policy, and authority"** (same authors, OpenKedge manuscript, Aug 2026) — extends the framing to serializability of agent commits.
- **The gap that's still open:** all three of these are formal/theoretical papers deriving quorum-threshold conditions mathematically — none of them appear to ship a working implementation with real LLM agents making real tool calls against a real side-effecting resource. That's an unusual amount of theory this early relative to empirical validation, which is actually a normal and legitimate opening for a systems-flavored semester project: **build and stress-test the Honest Quorum / EBFT model empirically** — implement their agreement-before-commit protocol with actual LLM agents, actual

adversarial/correlated prompting (shared poisoned tool or shared model lineage), and actual side-effecting actions (your banking/inventory example), and measure whether their theoretical thresholds hold up against real model behavior, or where they break.

MVP (weeks 1–4):

- Build a minimal harness with one side-effecting mock resource (a toy ledger/inventory service) that agents can call `transfer()` or `update_inventory()` against.
- Implement a lightweight PBFT-style three-phase protocol (propose → prepare → commit) that requires a quorum of agents to agree on the *exact action* before it's allowed to execute, instead of letting one agent act and hoping a later vote catches it.
- Inject Byzantine agents two ways: (a) independent adversarial system prompts arguing for a manipulated action, and (b) correlated faults where several agents share one poisoned tool/data source (this is where you can still legitimately draw on the "Consensus Trap" framing, but applied to actions instead of answers — that's your differentiator, not a rehash).
- Metrics that matter here and aren't in the existing papers: **unauthorized-commit rate** (bad actions that slip through), **false-block rate** (honest actions wrongly stopped), and **commit latency/LLM-call overhead** of requiring agreement vs. a single trusting agent. That last one is a genuine systems contribution — nobody in this space is looking at the cost/latency price of reliability.

Expansion directions once that's working:

1. Port the agreement layer onto Temporal or Dapr Agents so it's a real durable-execution primitive, not just a script — "trustworthy commit protocol for agent runtimes" is a much stronger systems-paper framing than another consensus-accuracy benchmark.
2. Explore the reliability/cost Pareto frontier — how many agents and rounds do you actually need for a target commit-safety level, as a function of fault rate and correlation.
3. Async version: relax the synchronous-round assumption entirely (agents propose/commit on their own schedule, messages can arrive out of order) — closer to how real agent runtimes behave and closer to classical partial-synchrony BFT theory than anything in the current literature.

- 

## Idea #2: Multi-agent codebase consistency

- What do we do if multiple agents want to work on the same files at once?
- Adding locks to ai agents
- AgentRoom: Concurrent Multi-Agent Coding in a CRDT-Backed Shared Workspace  
<https://openreview.net/forum?id=0aGLZqKJjt>
-

## Idea #2.5: Code Consistency Via Git

- Byzantine system to verify git commits
- Ensures that bad agents can't push through bad commits in open source projects

## Idea #3: Overload control for agentic workloads

**The gap.** Agent systems break classical overload control in ways nobody has measured rigorously. The failure mode is documented — but only in blog posts and one workshop position paper that is **simulation-only**. That paper's own framing of the problem is the best statement of it: metastability is the failure mode where retries, queues, timeouts, and load shedding turn into amplifiers, and under agents this gets worse because admission control can't tell an agent's first attempt from its fifth retry, and a rejection would usually cause the agent to escalate. Crucially, agents retry aggressively while mutating the query on each attempt — so deduplication, idempotency keys, and retry budgets all fail, because attempt #5 isn't textually the same request as attempt #1.

That last sentence is your research hypothesis. Every classical mechanism (dedupe windows, retry budgets, circuit breakers) assumes retries are *identical*. Agent retries are *semantically related but syntactically different*. Nobody has built or measured a system that handles this.

**What you build.** A proxy sitting between an agent framework and its downstream services (LLM API + tool/MCP servers) that:

1. Reconstructs the *retry tree* — linking mutated attempts back to a common logical request via lineage tracking (propagate a logical-request ID through the agent's context, not by matching request bodies).
2. Enforces overload control at the logical-request level: per-logical-request work budgets, backpressure that the agent framework actually understands (returning "reduce fan-out" rather than 429), and load shedding that sheds *whole logical requests* rather than random attempts.
3. Compare against: naive retry, exponential backoff + jitter, token-bucket retry budgets, circuit breakers.

**Why it's a good class project.** No GPUs needed — a small local model via vLLM/Ollama on one machine, or even a mocked LLM with realistic latency distributions for the load experiments. Genuinely distributed-systems (congestion control, backpressure, admission control, queueing). Clean hypothesis with a yes/no answer: *does lineage-aware overload control prevent metastable collapse in agentic workloads where request-level mechanisms fail?* And you can produce a real result — a goodput-vs-offered-load curve showing the collapse region and your mechanism eliminating it.

**Risk:** you need to induce metastability reliably. Build a deliberately capacity-limited tool server so you control the trigger point

## Blockchain AI Opensource Repo

- Dynamically allocate agents to reduce cost/overhead when making low-risk decisions, and increase safety during risky decisions
- Maybe a controlling agent?
- Measure tokens vs percent incorrect actions etc.
- Deal with faulty agents etc.
-